The Anatomy of Scent

Process Book

Alexandre Mourot (346365) & Gaël Conde Losada (329871)
COM-480 Data Visualization, EPFL, Spring 2026

Overview

We built a scrollytelling website that explores what makes a perfume attractive. Using 24,063 perfumes scraped from Fragrantica and over 2,000 eBay listings, the site walks the reader through perfume composition patterns, gender differences in fragrance profiles, temporal trends spanning five decades, and the relationship between scent and market price. Eight interactive D3.js visualizations drive the narrative, each revealing a different facet of the data.

Our target audience sits at the intersection of three groups. Perfume enthusiasts who want to see their passion through a data lens come first. Then there are data visualization lovers who appreciate well crafted interactive storytelling. Finally, we wanted to reach curious people who have never considered why bergamot shows up in nearly every fragrance they own. The goal was to surprise all three.

The motivation behind this project came from a simple observation: perfumery is an art form built on hidden structure. Every perfume follows a three layer architecture of top, middle, and base notes. Certain ingredients appear thousands of times while others remain rare signatures. We wanted to make that invisible structure visible, and to ask whether the patterns we found could explain why some fragrances succeed while others fade into obscurity.

The site was designed to feel like walking into a high end perfume boutique. Dark background, refined typography, gold accents. Not a dashboard. A story, told section by section as the reader scrolls.

Data and Processing

Datasets

Our primary dataset is the **Fragrantica Fragrance Dataset** from Kaggle, containing 24,063 perfumes across 18 columns. Each entry includes the perfume name, brand, gender category, release year, user rating, fragrance accords, and three columns listing top, middle, and base notes as comma separated text. The gender distribution splits roughly 47% women, 32% unisex, and 21% men.

We supplemented this with the **Perfume E-commerce Dataset 2024**, also from Kaggle, containing around 2,000 eBay listings split between men's and women's perfumes. This dataset adds pricing information, with men's listings ranging from $3 to $259 (mean $46) and women's from $2 to $300 (mean $40). Since the data comes from eBay, the pricing insights reflect primarily the U.S. market.

Data Quality

The Fragrantica data was generally clean but required meaningful preprocessing. Fragrance notes were stored as plain text strings that needed parsing into structured arrays. Similar note names required grouping into broader families (floral, woody, citrus, spicy, and others). Release year was absent for 8.5% of entries, and user ratings were only available for a subset of perfumes with enough votes. The eBay data posed a different challenge: brand names did not always match cleanly between the two datasets, requiring fuzzy matching.

Processing Pipeline

We processed everything in Python using pandas. The pipeline parsed note text into arrays, grouped 500+ unique note names into families, computed co-occurrence matrices for the chord diagram, built the Sankey flow from top through middle to base notes, and aggregated statistics by gender and decade. A key decision was to pre-compute the expensive aggregations in Python rather than doing them client side. The raw Fragrantica CSV alone is over 12 MB, far too heavy for a browser to process interactively.

The pipeline outputs eight JSON files, each tailored to a specific visualization:

- `perfumes.json` (full perfume records)
- `notes_stats.json` (frequency and rating per note)
- `notes_stats_enhanced.json` (with family groupings)
- `temporal_trends.json` (note families by decade)
- `price_data.json` (eBay pricing merged with composition)
- `accords_data.json` (accord frequency and ratings)
- `chord_data.json` (pre-computed co-occurrence matrix)
- `sankey_data.json` (top to middle to base note flows)

This separation of concerns meant the frontend only needed to fetch small, purpose built JSON files and bindthem directly to D3.js selections. No client side data wrangling.

Design Evolution

Initial Sketches

In Milestone 2, we sketched five core visualizations on paper. The opening section featured a beeswarm chart where each bubble represented a fragrance note, sized by frequency, with bubbles floating upward like scent molecules and clustered by note family. The second section placed side by side radar charts for men's and women's fragrance profiles. The third mapped note frequency against average rating in a bubble chart. A stacked area chart tracked note popularity across decades for the fourth section. The fifth combined eBay pricing with composition profiles in a scatter plot.

Two additional visualizations were planned as interactive sidebars: a chord diagram for note co-occurrence and a Sankey diagram for the top to middle to base flow.

Inspiration

The beeswarm concept drew directly from "Fragrance of Data," a project recognized at the Information is Beautiful Awards that used a similar particle aesthetic to represent scent data. Our scrollytelling structure was inspired by The Pudding, whose long form data stories use scroll position to control narrative pacing. We wanted the same feeling of the data revealing itself gradually as the reader moves through the page.

Visual Identity Decisions

Early on, we committed to a dark luxury aesthetic that would set the tone before any data appeared. The color scheme uses `#0a0a0a` as the primary background with `#c9a96e` gold accents, a palette that evokes the matte black and gold foil of high end perfume packaging. Typography pairs Cormorant Garamond for headings (an elegant serif with visible calligraphic DNA) with DM Sans for body text (a clean geometric sans serif that reads well at small sizes on dark backgrounds).

How the Sketches Evolved

> **BEESWARM (SECTION 1)**
>
> **Before:** Static bubble chart with manual positioning. All notes visible at once, creating visual clutter with 500+ circles competing for attention.
>
> **After:** D3 force simulation with scroll triggered transitions. The chart reveals notes progressively across four scroll steps: first musk alone, then bergamot, then the top 10, then the full constellation. Family clustering uses color encoding rather than spatial separation.

RADAR CHARTS (SECTION 2)

Before: Two separate static radar charts placed side by side. Eight axes representing the top note families.

After: Overlaid radar polygons that animate in sequence. Women's profile appears first, then men's overlays with a different color, making the comparison immediate. A toggle adds unisex as a third layer. The axes label the actual note families from the data rather than generic categories.

TIMELINE (SECTION 4)

Before: Simple stacked area chart from 1970 to 2020, showing all note families simultaneously.

After: Scroll controlled decade highlighting. As the reader progresses, each decade receives a focus state with annotations explaining the cultural context (the clean 1990s, the gourmand 2010s). The non-focused decades fade to lower opacity.

Key Technical Design Decisions

Scrollama for scroll management. We chose Scrollama over custom scroll event listeners because it wraps the IntersectionObserver API cleanly and handles the tricky math of determining which "step" element is currently active. This let us define narrative beats as simple HTML divs and trigger D3 transitions declaratively.

Lenis for smooth scrolling. Native browser scroll felt abrupt during testing, especially on sections where the visualization needed to hold steady while text scrolled past. Lenis adds momentum and easing (we used an exponential decay function with duration 1.2s) that makes the experience feel polished without fighting the browser.

Lazy initialization via IntersectionObserver. This was not in our original plan. We discovered a critical bug during development: several visualizations rendered at zero width because their SVG containers had no dimensions at initialization time. The root cause was CSS flex layout combined with sticky positioning. Containers below the fold had a computed width of 0 when the page loaded. Our fix was to defer initialization until an IntersectionObserver detects the container entering the viewport. This solved the sizing bug and also improved initial page load time since only the hero and first section initialize immediately.

Pre-computed chord and Sankey data. Our first attempt computed the note co-occurrence matrix in JavaScript by iterating through all 24,063 perfumes and cross-referencing their top, middle, and base notes. On a modern laptop this took over 4 seconds, which is unacceptable in a scroll driven experience. We moved the computation to a Python script (`compute_advanced_data.py`) that generates `chord_data.json` and `sankey_data.json` as static files. The browser now just fetches the pre-built matrix.

Shared utilities. Rather than duplicating tooltip logic, color scales, and note family mappings across eight visualization files, we centralized them in `main.js`. Each visualization module (e.g., `beeswarm.js`, `chord.js`) follows the same pattern: a single initialization function that takes a container selector, fetches its specific JSON file, and builds the SVG. This kept the codebase modular without introducing a build system.

> **DEEP DIVE SECTION (CHORD + SANKEY)**
>
> **Before:** Planned as floating sidebar panels that would appear alongside the main narrative sections.
>
> **After:** Dedicated section (Section 6) with a two column card layout. The sidebar concept broke on narrow viewports and created confusing z-index conflicts with the sticky scrollytelling containers. A dedicated section gave both visualizations room to breathe and simplified the layout logic.

> **HEATMAP (SECTION 7)**
>
> **Before:** Not in the original Milestone 2 sketches. We originally planned a perfume search feature as the final interactive element.
>
> **After:** We replaced the search feature with a full width interactive heatmap of note frequency by gender. The search feature required a much larger JSON payload (individual perfume records) and complex fuzzy matching logic. The heatmap proved more visually striking and told a clearer story about gender differences in fragrance composition.

Implementation and Challenges

Technology Stack

Layer	Technology	Purpose
Visualization	D3.js v7	All eight interactive charts
Scroll management	Scrollama	Triggering transitions on scroll position
Smooth scroll	Lenis 1.1	Momentum scrolling and easing
Sankey layout	d3-sankey 0.12	Node/link positioning for flow diagram
Data processing	Python 3, pandas	CSV parsing, aggregation, JSON export
Frontend	Vanilla HTML/CSS/JS	No framework, no build step
Hosting	GitHub Pages	Static deployment from `/docs`

We deliberately avoided frontend frameworks. A scrollytelling site with eight D3 visualizations does not need React's component model, and adding a build step would have complicated the GitHub Pages deployment. Every visualization lives in its own file under `js/visualizations/`, and `main.js` orchestrates initialization, scroll events, and shared utilities like color scales and tooltip rendering.

Architecture

The site follows a simple pattern. The HTML defines eight section containers, each with an `id` matching its visualization (e.g., `viz-beeswarm`, `viz-chord`). The five scrollytelling sections use a two column layout: a sticky graphic container on the left holds the SVG, while scrollable text steps on the right drive transitions. The remaining three visualizations (chord, sankey, heatmap) use simpler layouts since they do not need scroll linked narrative.

Data flows in one direction: Python scripts read the raw CSVs, compute aggregations, and write JSON files to `docs/data/`. The JavaScript never touches raw data. Each visualization module fetches its JSON, binds the data to SVG elements using D3's data join pattern, and registers its own interaction handlers.

Challenge 1: Note Taxonomy

The Fragrantica dataset contains over 500 unique note names with no standardized taxonomy. "White musk" and "musk" are semantically identical. "Pink pepper" belongs to the spicy family but could also be classified as fruity. We needed a consistent family mapping to drive the beeswarm clustering, radar chart axes, and timeline categories.

Our solution was a manually curated mapping table in the Python processing script. We grouped notes into seven families: floral, woody, citrus, spicy, fresh, sweet, and musky.

Ambiguous notes were assigned based on how perfumers typically classify them. This required reading actual perfumery references rather than guessing. The mapping is imperfect (some notes genuinely belong to multiple families) but it provides a consistent lens across all visualizations.

Challenge 2: SVG Sizing Bug

This was our most frustrating bug. Several visualizations rendered as invisible 0-pixel wide SVGs. The cause took hours to diagnose. In the scrollytelling layout, the sticky graphic container uses CSS `position: sticky` inside a flex parent. On initial page load, containers below the viewport had a computed width of 0 because the browser had not yet laid them out. Our D3 code read `container.clientWidth` during initialization and created an SVG with width 0.

The fix was lazy initialization. We wrapped each visualization's init function in an IntersectionObserver callback that fires only when the container first enters the viewport. By that point, the browser has computed the layout and `clientWidth` returns the correct value. This pattern also improved performance since visualizations below the fold no longer compete for resources during initial load.

Challenge 3: Chord and Sankey Performance

Computing note co-occurrence from 24,063 perfumes means iterating through every perfume, extracting all note pairs across top, middle, and base layers, and incrementing a matrix cell for each pair. In JavaScript, this loop took over 4 seconds on a mid-range laptop. For a scroll driven site where the user might reach the chord diagram section within 30 seconds of loading, a 4-second computation freeze is a dealbreaker.

We moved the entire computation to `src/compute_advanced_data.py`. The Python script generates a filtered co-occurrence matrix (keeping only the top 20 most connected notes to avoid visual clutter) and a Sankey flow dataset (top 15 flows from each layer). The resulting JSON files are under 50 KB each. The browser just fetches and renders.

Challenge 4: Dark Theme Contrast

The dark luxury theme created readability issues we did not anticipate. Our first version used grid lines at opacity 0.06, which were completely invisible on most monitors. Axis labels in the default D3 grey disappeared against `#0a0a0a`. Chart borders blended into the background.

We went through three rounds of contrast tuning. Grid lines moved to opacity 0.15. Axis text switched to `#9a9088` (our `text-secondary` variable). We added subtle `rgba(201, 169, 110, 0.12)` borders to card containers. The gold accent (`#c9a96e`) served double duty as a highlight color and as a border/separator that is visible without being harsh.

Challenge 5: Responsive Design

Scrollytelling fundamentally assumes a wide viewport. The pattern relies on side by side layout: sticky graphic on one side, scrolling text on the other. On a phone screen, this breaks. The graphic shrinks to an unusable size or the text overlaps the chart.

We implemented CSS media queries that stack the layout vertically on screens narrower than 768px. The graphic sits above and scrolls with the page instead of staying sticky. Text steps appear below each visualization. This is a compromise: the scroll triggered transitions still fire, but the user sees the chart above rather than beside the text. It works, but the experience is noticeably better on desktop. Given more time, we would have built a separate mobile layout with smaller, simplified chart versions.

Deployment

The site deploys from the `/docs` directory via GitHub Pages. Because we have no build step, deployment is just a git push. The JSON data files live in `docs/data/` alongside the HTML, CSS, and JS. Total deployed size is under 15 MB, with the largest file being `perfumes.json` at about 8 MB (loaded only when needed for search or filtering, not on initial render).

Final Visualizations

The website presents eight visualizations across seven sections. Each one answers a different question about perfume data.

Visualization	Type	What It Shows
1. Building Blocks	Beeswarm / force layout	Frequency of each fragrance note across 24K perfumes, clustered by family
2. His & Hers	Radar chart	Note family profiles compared across women's, men's, and unisex perfumes
3. Ratings Game	Bubble chart	Note frequency vs. average user rating, sized by perfume count
4. Fifty Years	Stacked area / timeline	Shifts in note family popularity from the 1970s to the 2020s
5. Price of Scent	Scatter plot	eBay pricing patterns across gender and composition profiles
6a. Note Connections	Chord diagram	Which notes most frequently co-occur in the same perfume
6b. Flow of Fragrance	Sankey diagram	How notes flow from top layer through middle to base
7. Full Picture	Interactive heatmap	Note frequency breakdown by gender category

Key Insights Discovered

> ***Musk and bergamot are universal.*** *These two notes appear in both men's and women's top 5 most frequent ingredients. They are the invisible foundation that modern perfumery shares across all gender categories.*

> ***Rarity correlates with higher ratings.*** *Notes like pink pepper and tonka bean appear in relatively few perfumes but carry some of the highest average user scores. The bubble chart makes this pattern immediately visible in the upper left quadrant.*

> ***Gender differences are real but concentrated.*** *Floral notes (jasmine, rose, lily of the valley) dominate women's perfumes while woody and spicy notes (patchouli, cedar, vetiver) dominate men's. But the overlap is larger than most people expect. The radar chart overlay makes this clear.*

> ***Perfume composition has shifted dramatically.*** *The timeline reveals a clear arc from aldehydic glamour in the 1970s through the clean aquatic 1990s to the gourmand and oud explosion of the 2010s and 2020s. Fragrance trends mirror broader cultural movements.*

> ***Price does not strongly correlate with composition complexity.*** *Premium brands use slightly more complex note structures, but the scatter plot shows no clean linear relationship. Brand cachet and marketing appear to drive pricing more than raw ingredient sophistication.*

Peer Assessment

We split the work equally (50/50) with each team member owning distinct deliverables.

Area	Alexandre Mourot	Gaël Conde Losada
Data	Processing pipeline, Python scripts, JSON generation, note taxonomy mapping	Exploratory data analysis, dataset selection, data quality assessment
Architecture	Website structure, scrollytelling setup with Scrollama and Lenis, lazy initialization system	Milestone reports (M1, M2), process book, testing across browsers
Visualizations	Beeswarm, radar charts, bubble chart, interactive heatmap	Chord diagram, Sankey diagram, timeline chart, price analysis
Design	CSS dark theme, responsive layout, typography, color system	Content writing for scroll steps, narrative structure, user flow

What We Would Do Differently

More user testing. We showed the site to a handful of classmates in the final week, but earlier feedback would have caught issues like the invisible grid lines and confusing axis labels sooner. Formal usability testing with 5 to 10 participants at the midpoint would have improved the final result noticeably.

An interactive perfume search. Our Milestone 2 stretch goals included a feature where the reader could type a perfume name and see its note profile as a radar chart. We dropped this in favor of the heatmap because it was simpler to build, but a search feature would have made the site feel more personal. Readers want to find their own perfume in the data.

Better mobile experience. The responsive layout works but feels like a fallback rather than a designed experience. A dedicated mobile version with simplified charts (perhaps static images with key annotations instead of full D3 interactivity) would serve phone users better. Scrollytelling is inherently a desktop pattern, and we should have planned for that constraint from the beginning.

Dataset enrichment. The eBay pricing data covers only about 2,000 listings, making the price analysis section our weakest link statistically. A larger e-commerce dataset or direct integration with retailer APIs would have strengthened that section considerably.

Conclusion

The Anatomy of Scent transforms 24,063 rows of perfume data into a story anyone can follow. Through eight interactive visualizations and a scroll driven narrative, we made the hidden structure of perfumery visible and explorable. The project taught us that the hardest part of data visualization is not the code or the data. It is choosing what to show, when to show it, and how much to let the reader discover on their own.